



Software Testing

CHAPTER 3. SOFTWARE TESTING APPROACHES AND TECHNIQUES

Session 2

BLACK BOX TESTING



Contents

1

What is black box testing?

2

The black box testing techniques

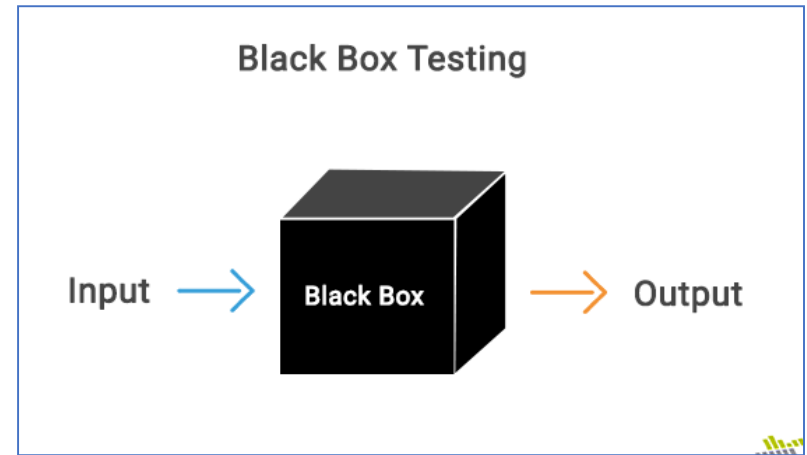


Black box testing

❖ As known as:

- Specification-based testing
- Behavior-based testing

❖ Black box testing is a software testing method in which the functionalities of software applications are tested without having knowledge of internal code structure, implementation details and internal paths.





Black box testing

- ❖ The tester focuses on **what the software does**, not how it does it.
- ❖ Testing based on software requirements and specifications.
- ❖ Black box testing consist of
 - Functional testing
 - Non-functional testing



Black box testing

❖ Common techniques of black box testing:

1. Equivalence Class Testing
2. Boundary Value Testing
3. Combinatorial Testing/Pairwise Testing/All-Pairs Testing
4. State Transition Testing
5. Decision Tables Testing
6. Use Case Testing

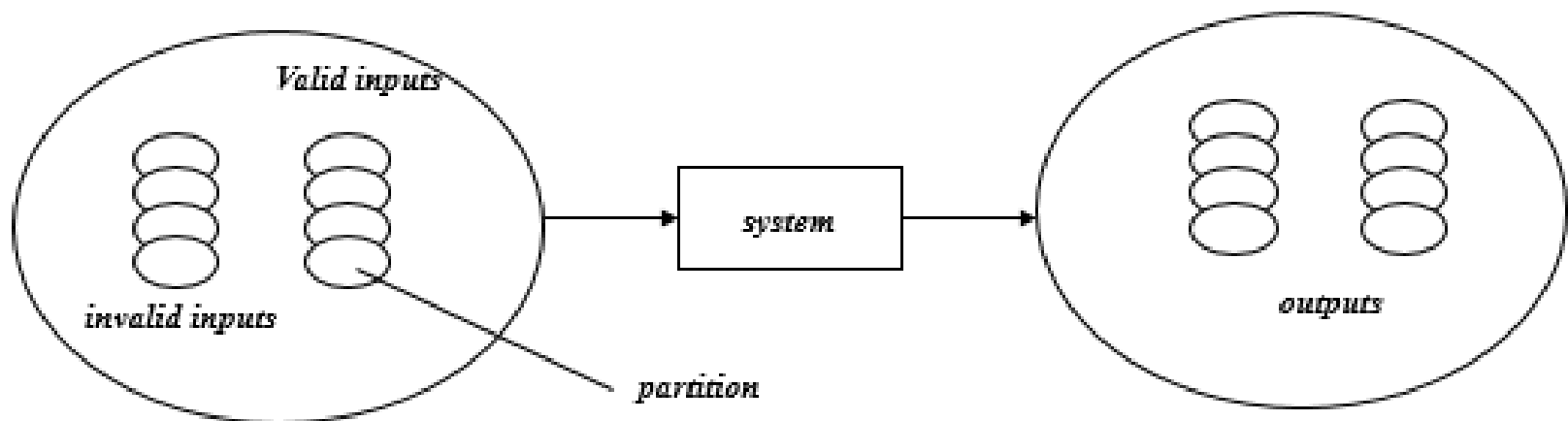


1. Equivalence Class Testing

- ❖ It is also known as Equivalence Partitioning
- ❖ Idea:
 - Divide or partition the input domain of a program into classes of data
 - Derive test cases based on these partitions
- ❖ Goal:
 - to reduce the total number of test cases to a finite set of testable test cases, still covering maximum requirements.

1. Equivalence Class Testing

- ❖ The system will handle all the test input variations within a partition in the same way.
 - If one of the input condition passes, then all other input conditions within the partition will pass as well.
 - If one of the input conditions fails, then all other input conditions within the partition will fail as well.





Examples 1

- ❖ Assume, we have to test a field which accepts Age 18 – 56

AGE *Accepts value 18 to 56

EQUIVALENCE PARTITIONING		
Invalid	Valid	Invalid
≤ 17	18-56	≥ 57

- ❖ Valid Class: 18 – 56

- ❖ Invalid Class 1: ≤ 17

- ❖ Invalid Class 2: ≥ 57

=> There are 3 test cases using equivalence class testing

Test Case	Input	Expected result
1	20	Accept
2	15	Reject
3	60	Reject



Example 2

- ❖ A program takes 3 integer inputs representing the lengths of the sides of a triangle.
- ❖ The program determines if the side values represent an equilateral triangle, a scalene triangle, an isosceles triangle, or whether the inputs would not represent a triangle.



Example 2

❖ In order for 3 integers a , b , and c to be the sides of a triangle, we must have:

✓ $a + b > c$

✓ $a + c > b$

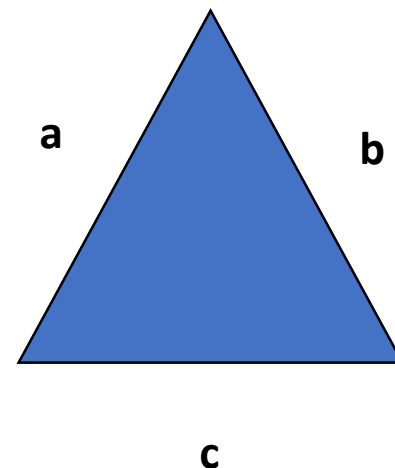
✓ $b + c > a$

❖ A triangle is:

✓ Equilateral if all 3 sides are equal

✓ Isosceles if 2 sides are equal

✓ Scalene if no two sides are equal





Example 2

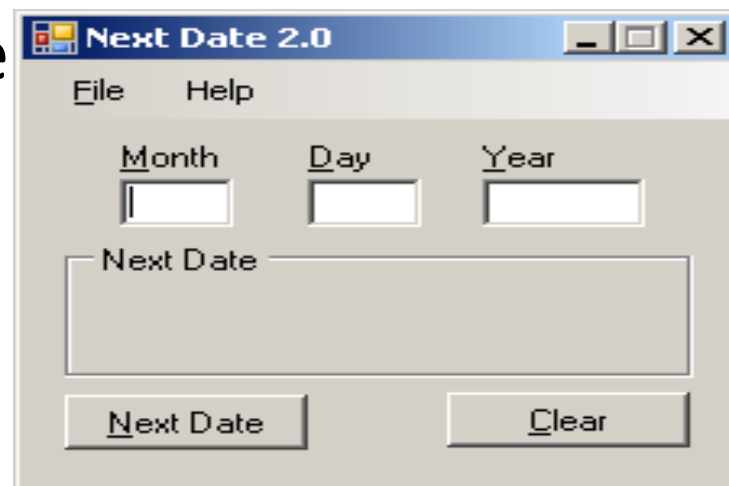
- ❖ Assume that each side length is between 1 and 200
- ❖ We may define the input test data by defining the equivalence class through the 5 output groups:
 - Error Input: $a \leq 0 \ || \ b \leq 0 \ || \ c \leq 0$
 - Not Triangle: $a + b \leq c \ || \ b + c \leq a \ || \ a + c \leq b$
 - Equilateral: $a == b \ \&\& \ b == c$
 - Scalene: $a != b \ \&\& \ b != c \ \&\& \ a != c$
 - Isosceles: $(a == b \ \&\& \ b != c) \ || \ (b == c \ \&\& \ c != a) \ || \ (a == c \ \&\& \ a != b)$

Test Case	a	b	c	Expected Output
OE1	5	5	5	Equilateral
OE2	2	2	3	Isosceles
OE3	3	4	5	Scalene
OE4	4	1	2	Not a Triangle



Exercise

❖ This program reads in a date in a certain format and prints out the next day's date



- For example, an input of 03-31-1998 gives an output of 04-01-1998.
- The year is constrained to lie between 1812 and 2012 inclusive

=> Define equivalence classes , test conditions (of input data) using equivalent partitioning technique



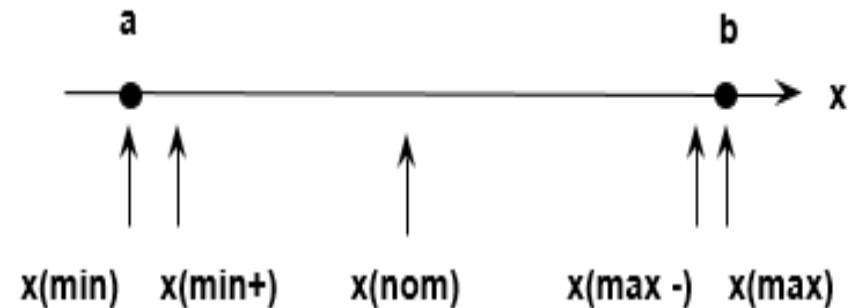
2. Boundary Value testing

- ❖ It is testing at the boundaries between partitions
- ❖ It is known as Boundary Value Analysis (BVA)
- ❖ To find the errors at boundaries of input domain (tests the behavior of a program at the input boundaries).
- ❖ The minimum and maximum values of a partition are its boundary values.
- ❖ Boundary value analysis can perform at all test levels, and its primarily used for a range of numbers, dates, and time.

2. Boundary Value testing

❖ The basic idea of boundary value analysis (BVA) is to use input variable values at

- minimum,
- just above the minimum,
- at a nominal value,
- just below the maximum,
- at the maximum





Example 1:

- ❖ Assume that we have to test a text field (Age) which accepts age from 18 to 56 as below:

Age:

Enter Age

* Accepts any value from 18 to 56

Boundary Value Analysis

Invalid (min -1)	Valid (min, +min, -max, max)	Invalid (max+1)
17	18,19,55,56	57



Example 2

❖ Boundary Value Testing for the Triangle Program

- Side lengths are integers ranging from 1 to 200
- Typical boundary value analysis requires 5 values for each variable: min, min+, nominal value, max-, max
- Input Boundary Value testing for 3 variables requires **5x3 = 15 test cases.**
- Our values: 1, 2, 100, 199, 200



Example 2

❖ Boundary Value Test Cases for the Triangle problems

Case	a	b	c	Expected Output
1	100	100	1	Isosceles
2	100	100	2	Isosceles
3	100	100	100	Equilateral
4	100	100	199	Isosceles
5	100	100	200	Not a Triangle
6	100	1	100	Isosceles
7	100	2	100	Isosceles
8	100	100	100	Equilateral
9	100	199	100	Isosceles
10	100	200	100	Not a Triangle
11	1	100	100	Isosceles
12	2	100	100	Isosceles
13	100	100	100	Equilateral
14	199	100	100	Isosceles
15	200	100	100	Not a Triangle



Exercise

❖ Bank has different charges depending on the transaction done:

- + 5% of the amount for transaction less than or equal 1000 USD

- + 6% of the amount for transaction more than 1000 USD and less than or equal 2000 USD

- + 7% of the amount for transaction more than 2000 USD

=> Lets create the set of TCs by BVA technique!



3. Pairwise Testing

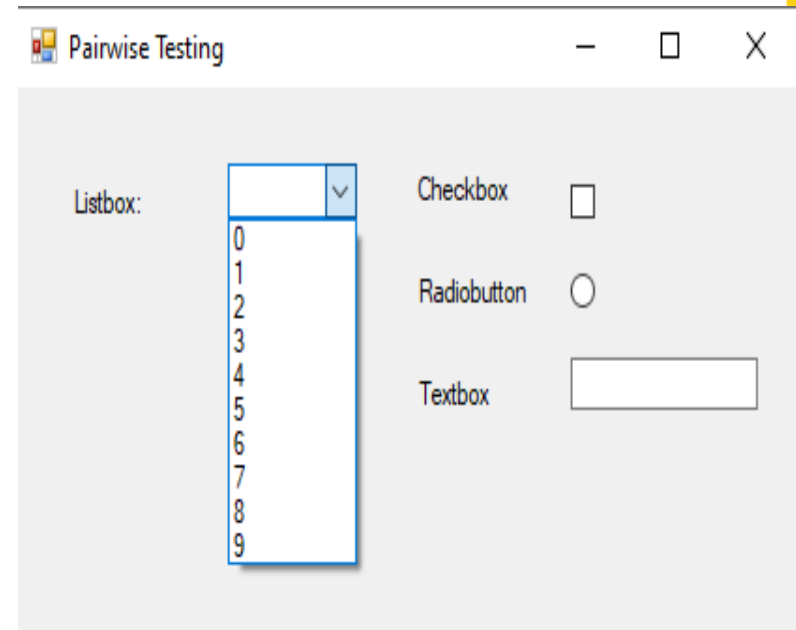
- ❖ Is known as All-Pairs Testing or Combinatorial
- ❖ is a black-box test design technique in which test cases are designed to execute all possible discrete **combinations of each pair of input parameters**.
- ❖ Pairwise-generated test suites cover all combinations of two therefore are much smaller than exhaustive ones yet still very effective in finding defects



Example

❖ An GUI application

- List Box - 0,1,2,3,4,5,6,7,8,9
- Check Box - Checked or Unchecked
- Radio Button - ON or OFF
- Text Box - Any Value between 1 and 100





Example

❖ Exhaustive combination is calculated.

✓ List Box = 10

✓ Check Box = 2

✓ Radio Button = 2

✓ Text Box = 100

=> Total Number of Test Cases using Cartesian Method :
 $10 * 2 * 2 * 100 = 4000$

=> Total Number of Test Cases including Negative Cases
will be > 4000

=> the idea is to bring down the number of test cases.



Example

Listbox(2)	Checkbox(2)	Radio(2)	Textbox(3)
0	Check	On	Valid Int
Others	Uncheck	Off	Invalid Int
			AlphaSpecialChar

❖ We can consider:

- List box values as 0 and others.
- Radio button and check box values cannot be reduced (ON or OFF).
- The Text box values can be reduced into three inputs (Valid Integer, Invalid Integer, Alpha-Special Character).

=> The number of cases using software testing technique, $2*2*2*3 = 24$ (including negative cases).



Example

❖ Reduce the combination further into All-pairs technique:

<i>Textbox</i>	<i>Listbox</i>	<i>Checkbox</i>	<i>Radio</i>
ValidInt	0	Check	On
ValidInt	Others	Uncheck	Off
InvalidInt	0	Uncheck	On
InvalidInt	Others	Check	Off
AlphaSpecialChar	0	Check	Off
AlphaSpecialChar	Others	Uncheck	On

=>6 Test Cases.



3. Pairwise Testing

❖ We can use Pairwise Testing tools to effectively automate the Test Case Design process by generating a compact set of parameter value choices as the desired Test Cases:

- [PICT](#) – ‘Pairwise Independent Combinatorial Testing’, provided by Microsoft Corp.
- [IBM FoCuS](#) – ‘Functional Coverage Unified Solution’, provided by IBM.
- [ACTS](#) – ‘Advanced Combinatorial Testing System’, provided by NIST, an agency of the US Government.
- [Pairwise](#) by Inductive AS
- [VPTag](#) free All-Pair Testing Tool



Exercise

❖ Pair-Wise Testing for a GUI application

A screenshot of a GUI dropdown menu labeled 'Seat type'. The menu is open, showing four options: 'Berth', 'Coupe' (which is highlighted with a dark background), and 'Lux'. There are up and down arrow buttons on the right side of the menu.

Provide additional information about ticket

Bed linen ☐

Tea ☐

Gypsies ☐

Demobeers ☐

Next



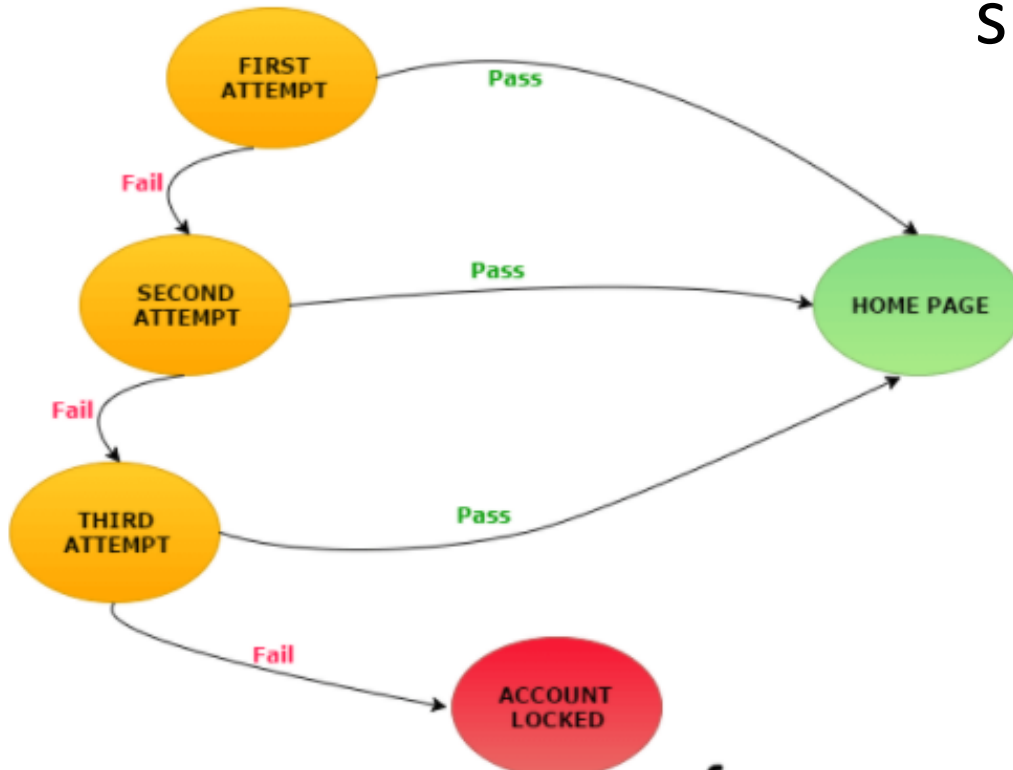
4. State Transition Testing

- ❖ State transition testing is performed to check the various states of scenario/system and possible transition between them.
- ❖ is helpful where you need to test different system transitions.



Example 1

❖ Login page of an application which locks the user name after three wrong attempts of password.



- The state diagram shows:

- 5 states:

- First attempt (S1)
- Second attempt (S2)
- Third attempt (S3)
- Home Page (S4)
- Account locked (S5)

- Events:

- Correct password
- Incorrect password



Example 1

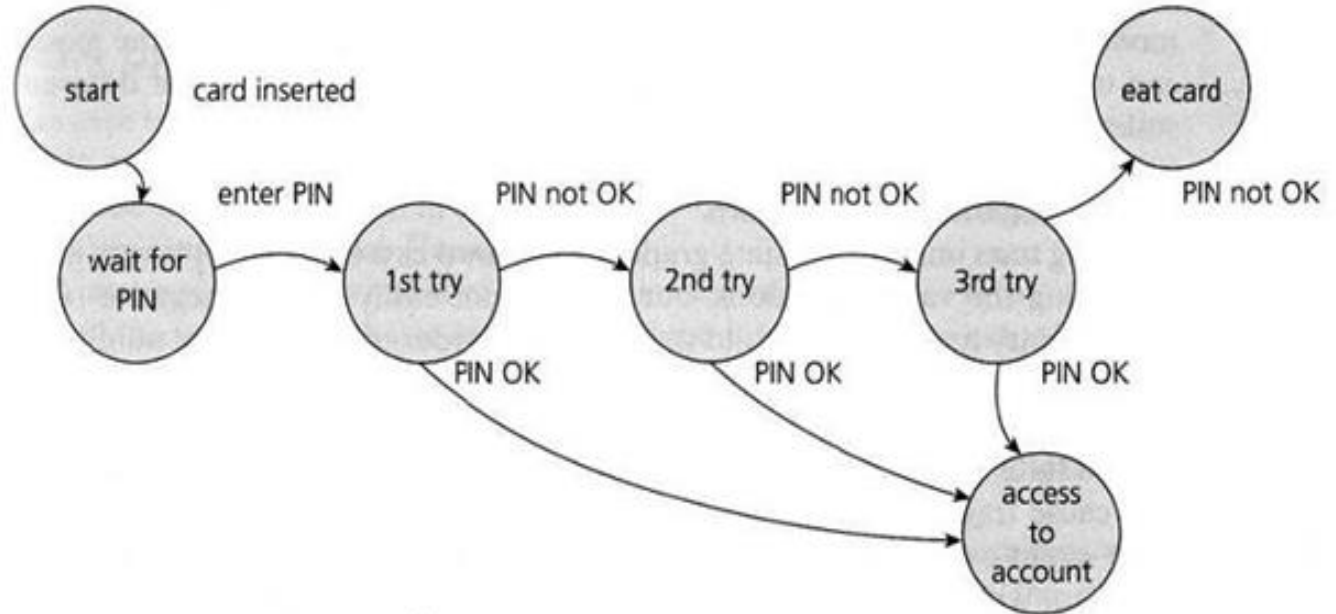
❖ Determine the states, input data and output data.

STATE	LOGIN	CORRENT PASSWORD	INCORRECT PASSWORD
S1	First Attempt	S4	S2
S2	Second Attempt	S4	S3
S3	Third Attempt	S4	S5
S4	Home Page		
S5	Display a message as "Account Locked, please consult Administrator"		



Example 2

- ❖ The state diagram shows an example of entering a Personal Identity Number (PIN) to a bank account.
- ❖ The states are shown as circles, the transitions as lines with arrows and the events as the text near the transitions.





Example 2

❖ The state diagram shows:

✓ **States**: 7 states:

S1:Start, S2:Wait for Pin, S3: 1st try, S4: 2nd Try,
S5: 3rd Try, S6: access to account, S7: eat card

✓ **Events**: 4 events

Event 1:Card inserted

Event 2: Enter Pin,

Event 3: Pin OK

Event 4: Pin not OK

✓ **Actions** : (not shown in the above example) could be :
Messages on the screen – error or otherwise.



Example 2

❖ The State Table for the PIN example can be simplified as below:

	Event 1 (Insert card)	Event 2 (Enter Pin)	Event 3 (Pin OK)	Event 4 (Pin not OK)
S1:Start	S2	-	-	-
S2:Wait for Pin	-	S3	-	-
S3: 1st try	-	-	S6	S4
S4: 2nd Try	-	-	S6	S5
S5: 3rd Try	-	-	S6	S7
S6: access to account	-	-	-	-
S7: eat card	S1	-	-	-



Exercise

- ❖ A website shopping basket starts out empty. As purchases are selected, they are added to the shopping basket. Items can also be removed from the basket.
- ❖ When the customer decides to check out, a summary of the items and the total cost are shown. Customer states if the information is ok
- ❖ If the contents and the price are OK, then the customer will be redirected to the payment system. Otherwise, you go back to shopping
 - Produce a state diagram
 - Define a state table which to cover all transitions



4. Decision Table Testing

- ❖ Decision table testing is black box test design technique to **determine the test scenarios** for complex business logic.
- ❖ We can apply Equivalence Partitioning and Boundary Value Analysis techniques to only **specific conditions or inputs**.
- ❖ Helps testers to search the **effects of combinations of different inputs**.



4. Decision Table Testing

❖ Decision table components:

- Conditions:

- ✓ Inputs are interpreted as *conditions* (Dashes represent *don't care* conditions)
- ✓ Condition entries binary values, they are called **limited entry table**.
- ✓ Condition entries have more than two values, they are called **extended entry table**.

- Actions

- ✓ Outputs are interpreted as *actions*

- Rules:

- ✓ The columns in a table are *rules* — they show which actions result from which conditions
- ✓ Every rule then becomes a test case



4. Decision Table Testing

❖ A decision table has 4 portions:

1. Condition stub
2. Condition entry
3. Action stubs
4. Action entry



4. Decision Table Example

rules

<i>Stub</i>	<i>Rule 1</i>	<i>Rule 2</i>	<i>Rule 3</i>	<i>Rule 4</i>	<i>Rule 5</i>	<i>Rule 6</i>
c1	T	T	T	F	-	T
c2	F	T	T	T	-	T
c3	T	T	-	T	T	T
c4	T	F	F	T	T	T
a1	X	X		X	X	X
a2		X				
a3	X			X		
a4			X			X

conditions

values of
conditions

actions

actions taken

Read a Decision Table by columns of rules :
R6 says when all conditions are T, then actions a1, a4 occur



4. Decision Table

❖ A Redundant Decision Table:

c1:	1-4 T	5 F	6 F	7 F	8 F	9 T
c2:	--	T	T	F	F	F
c3:	--	T	F	T	F	F
a1:	X	X	X	--	--	X
a2:	--	X	X	X	--	--
a3:	X	--	X	X	X	X

- Rule 9 is identical to Rule 4 (T, F, F)
- Since the action entries for rules 4 and 9 are identical, there is no ambiguity, just redundancy

4. Decision Table

❖ An Inconsistent Decision Table

Conditions	1-4	5	6	7	8	9
c1:	T	F	F	F	F	T
c2:	--	T	T	F	F	F
c3:	--	T	F	T	F	F
a1:	X	X	X	--	--	--
a2:	--	X	X	X	--	X
a3:	X	--	X	X	X	--

- Rule 9 is identical to Rule 4 (T, F, F)
- Since the action entries for rules 4 and 9 are different, there is ambiguity => the table is inconsistent, and the inconsistency implies non-determinism — can't tell which rule to apply!



4. Decision Table

❖ Procedure for Decision Table Testing:

1. Determine conditions and actions.
2. Develop the Decision Table, watching for
 - ✓ Completeness
 - ✓ Don't care entries
 - ✓ Redundant and inconsistent rules
3. Each rule defines a test case



Example

- ❖ A university computer system allows students an allocation of disc space depending on their projects.
- ❖ If they have used all their allotted space, they are only allowed restricted access, i.e. to delete files, not to create them.
- ❖ This is assuming they have logged on with a valid username and password.

What are the input and output conditions?



Example

❖ Input Conditions: each entry in the table may be either 'T' for true, 'F' for false.

Input Conditions

Valid username	T	T	T	T	F	F	F	F
Valid password	T	T	F	F	T	T	F	F
Account in credit	T	F	T	F	T	F	T	F



Example

❖ Rationalise input combinations

- Some combinations may be impossible or not of interest
- Some combinations may be 'equivalent'
- use a hyphen to denote "don't care"

Input Conditions

Valid username	F	T	T	T
Valid password	-	F	T	T
Account in credit	-	-	F	T



Example

- ❖ Determine the expected output conditions for each combination of input conditions
- ❖ Each column is at least one test case

Input Conditions				
Valid username	F	T	T	T
Valid password	-	F	T	T
Account in credit	-	-	F	T
Output Conditions				
Login accepted	F	F	T	T
Restricted access	-	-	T	F
Tags	A	B	C	D





Example

❖ Design test cases

TC	Description	Expected Outcome	Tag
1	Username BrbU	Invalid username	C
2	Username username toolong	Invalid username	A
3	Username BobU Password abcd	Invalid password	B
4	Valid user, no disc space	Restricted access	C
5	Valid user with disc space	Unrestricted access	D



Exercise

Create decision tables and some test cases for the following problems:

❖ Ex1: Login function:

- User enters their user ID, then user enters their password
- If the user enters the incorrect password three times, the account is locked
- Display an error message if access to system is granted with valid account otherwise, allow user to login

❖ Ex2: NextDate function

❖ Ex3: Triangle problem



5. Use case testing

- ❖ Use case testing is a Black Box testing technique helps identify test cases that cover the entire system, on a transaction by transaction basis from start to finish.
- ❖ It is helpful for deriving test cases for integration testing, system testing and acceptance testing.
- ❖ Each use case describes the interaction between the actor and the system to achieve a specific task



Example

❖ Consider the 'Show Student Marks' case, in a School Management System.

- **Use case Name:** Show Student Marks
- **Actors:** Students, Teachers, Parents
- **Pre-Condition:**
 - ✓ The system must be connected to the network.
 - ✓ Actors must have a 'Student ID'.



Example

❖ Use case testing Show Student Marks

Main Scenario	Serial Number	Steps
A: Actor/ S: System	1	Enter Student Name
	2	System Validates Student Name
	3	Enter Student ID
	4	System Validates Student ID
	5	System shows Student Marks
Extensions	3a	Invalid Student ID S: Shows an error message
	3b	Invalid Student ID entered 4 times. S: Application Closes



Example

❖ Design test cases

Test Cases	Steps	Expected Result
A	View Student Mark List 1 -Normal Flow	
1	Enter Student Name	User can enter Student name
2	Enter Student ID	User can enter Student ID
3	Click on View Mark	System displays Student Marks
B	View Student Mark List 2-Invalid ID	
1	Repeat steps 1 and 2 of View Student Mark List 1	
2	Enter Student ID	System displays Error message



Exercise

❖ Applying Use case testing for:

Ex1: Login functionality of a Web application

The image shows a login form with two input fields: 'Email' and 'Password'. The 'Email' field has an envelope icon on the right, and the 'Password' field has a lock icon on the right. Below the fields is a green 'Log in' button. A red speech bubble points to the 'Password' field, containing the text: 'Invalid password entered more than 4 times, IP address is banned'.



ĐẠI HỌC ĐÀ NẴNG
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG VIỆT - HÀN
Vietnam - Korea University of Information and Communication Technology

Thank You !